

Guía de estudio — Fundamentos matemáticos, descripción formal y pseudocódigo de TODOS los algoritmos

Basado en tu notebook (supervisado, no supervisado y adicionales). Lo estructuré en formato que te puede servir para el inciso 3. **Algoritmo y código de teoría** del examen.

1. Regresión Lineal

Complejidad

Tiempo: $O(n \cdot d \cdot \text{iteraciones})$ Espacio: $O(d)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo REGRESION_LINEAL_DESCENSO
Entrada: X, y, alpha, iteraciones
Complejidad tiempo:  $O(n \cdot d \cdot \text{iteraciones})$ 
Complejidad espacio:  $O(d)$ 

Inicializar w, b
Para t = 1 hasta iteraciones
    y_pred <- Xw + b
    dw <-  $(1/n)X^T(y_{\text{pred}} - y)$ 
    db <-  $(1/n)\text{sum}(y_{\text{pred}} - y)$ 
    w <- w - alpha * dw
    b <- b - alpha * db
FinPara
Retornar modelo
```

Fundamentos matemáticos

Fundamentos matemáticos

Modelo:

$$\hat{y} = wX + b$$

Función de costo (Error Cuadrático Medio):

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Gradientes:

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\hat{y} - y)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum (\hat{y} - y)$$

Actualización:

$$w = w - \alpha dw$$

$$b = b - \alpha db$$

Descripción formal

Algoritmo supervisado de regresión que ajusta una recta o hiperplano minimizando el error entre valores reales y predichos mediante descenso de gradiente.

Pseudocódigo

```
Inicializar w,b
Repetir iteraciones:
    calcular y_pred = Xw+b
    calcular gradientes dw,db
    actualizar w,b
Retornar modelo
```

2. Regresión Logística

Complejidad

Tiempo: $O(n \cdot d \cdot \text{iteraciones})$ Espacio: $O(d)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo REGRESION_LOGISTICA
Entrada: X,y
Complejidad tiempo:  $O(n \cdot d \cdot \text{iteraciones})$ 
Complejidad espacio:  $O(d)$ 

Inicializar pesos
Repetir:
     $z = Xw + b$ 
     $\hat{y} = \text{sigmoid}(z)$ 
    calcular gradientes
    actualizar pesos
```

```
Hasta convergencia
Si  $\hat{y} > 0.5$  clase 1
```

Fundamentos

Fundamentos

Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Predicción:

$$P(y=1 | x) = \sigma(wx + b)$$

Log-loss:

$$J = -\frac{1}{m} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Descripción

Clasificador supervisado binario que estima probabilidades y usa umbral 0.5.

Pseudocódigo

```
Inicializar pesos
Repetir:
     $z = Xw + b$ 
     $\hat{y} = \text{sigmoid}(z)$ 
    calcular gradientes
    actualizar pesos
Si  $\hat{y} > 0.5$  clase 1
```

3. SVM (Support Vector Machine)

Complejidad

Tiempo: $O(n^2 \cdot d)$ aprox. Espacio: $O(n)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo SVM_LINEAL
Entrada: datos etiquetados
```

```
Complejidad tiempo:  $O(n^2 \cdot d)$ 
Complejidad espacio:  $O(n)$ 

Inicializar w,b
Para cada iteración
  Para cada dato:
    calcular margen
    Si margen  $\geq 1$ 
      actualizar regularización
    Si no
      actualizar con hinge loss
Fin
```

Fundamentos

Fundamentos

Hinge Loss:

$$L = \max(0, 1 - y(wx - b))$$

Función objetivo:

$$J = \lambda \|w\|^2 + L$$

Restricción:

$$y(wx - b) \geq 1$$

Descripción

Busca hiperplano que maximice margen entre clases.

Pseudocódigo

```
Inicializar w,b
Para cada iteración
  Para cada dato:
    calcular margen
    Si margen  $\geq 1$ :
      actualizar regularización
    Si no:
      actualizar con hinge loss
Retornar modelo
```

4. KNN

Complejidad

Entrenamiento: $O(1)$ Predicción: $O(n \cdot d)$ Espacio: $O(n)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo KNN
Entrada: dataset, nuevo_punto, k
Complejidad tiempo:  $O(n \cdot d)$ 
Complejidad espacio:  $O(n)$ 

Para cada punto i
    calcular distancia
FinPara
Ordenar distancias
Tomar k vecinos
Votar mayoría
Retornar clase
```

Fundamentos

Fundamentos

Distancia euclidiana:

$$d(x_i, x_j) = \sqrt{\sum (x_i - x_j)^2}$$

Clase:

$$\hat{y} = \text{moda}(k \text{ vecinos})$$

Descripción

Lazy learning. No entrena; memoriza datos.

Pseudocódigo

```
Guardar datos entrenamiento
Para nuevo punto:
    calcular distancias
    tomar k vecinos más cercanos
    votar mayoría
    retornar clase
```

5. Árbol de Decisión

Complejidad

Tiempo: $O(n \cdot d \cdot \log n)$ Espacio: $O(n)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo ID3
Entrada: dataset, atributos
Complejidad tiempo:  $O(n \cdot d \cdot \log n)$ 
Complejidad espacio:  $O(n)$ 

Si nodo puro retornar hoja
Calcular ganancia por atributo
Elegir máxima ganancia
Dividir datos
Llamar ID3 recursivo
```

Fundamentos

Fundamentos

Entropía:

$$H(S) = -\sum p_i \log_2(p_i)$$

Ganancia:

$$IG = H(\text{padre}) - H(\text{hijos})$$

Descripción

Divide datos por mejores reglas feature-umbral.

Pseudocódigo

```
Si nodo puro o profundidad máxima:
    retornar hoja
Buscar mejor split
Partir datos
Construir ramas recursivamente
```

6. Random Forest

Complejidad

Tiempo: $O(t \cdot n \cdot d \cdot \log n)$ Espacio: $O(t \cdot n)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo RANDOM_FOREST
Entrada: datos, t árboles
Complejidad tiempo:  $O(t \cdot n \cdot d \cdot \log n)$ 
Complejidad espacio:  $O(t \cdot n)$ 

Para t árboles
    bootstrap sample
    entrenar árbol
FinPara
Votación mayoritaria
```

Fundamentos

Fundamentos

Bagging:

Bootstrap + votación.

$$\hat{y} = \arg\max_c \sum I(h_i(x) = c)$$

Descripción

Conjunto de árboles entrenados con muestras aleatorias.

Pseudocódigo

```
Para t árboles:
    crear muestra bootstrap
    entrenar árbol
Guardar árboles
Predicción:
    votación mayoritaria
```

7. K-Means

Complejidad

Tiempo: $O(n \cdot k \cdot \text{iter})$ Espacio: $O(n+k)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo KMEANS
Entrada: puntos, K
Complejidad tiempo:  $O(n \cdot k \cdot \text{iter})$ 
Complejidad espacio:  $O(n+k)$ 

Inicializar centroides
Repetir
    asignar puntos
    recalcular centroides
Hasta convergencia
```

Fundamentos

Fundamentos

Minimiza inercia:

$$J = \sum ||x_i - \mu_j||^2$$

Centroides:

$$\mu_j = \frac{1}{|C_j|} \sum x_i$$

Descripción

Agrupar datos por proximidad a centroides.

Pseudocódigo

```
Inicializar K centroides
Repetir:
    asignar puntos a centroides cercanos
    recalcular centroides
hasta convergencia
```


8. DBSCAN

Complejidad

Tiempo: $O(n^2)$ Espacio: $O(n)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo DBSCAN
Entrada: eps,minPts
Complejidad tiempo:  $O(n^2)$ 
Complejidad espacio:  $O(n)$ 
```

```
Para cada punto
  hallar vecinos
  si vecinos < minPts ruido
  si no expandir cluster
Fin
```

Fundamentos

Fundamentos

Punto núcleo:

$|N_{\epsilon}(p)| \geq \text{minPts}$

Descripción

Agrupar por densidad; detecta ruido.

Pseudocódigo

```
Para cada punto:
  si no visitado:
    hallar vecinos
    si pocos vecinos: ruido
    si núcleo: expandir cluster
```

9. Árbol de Decisión Regresor

Fundamentos

Varianza ponderada:

$$MSE = \frac{n_L \text{Var}(y_L) + n_R \text{Var}(y_R)}{n}$$

Descripción

Como árbol clasificación pero predice promedio continuo.

Pseudocódigo

```
Buscar split que minimice MSE
Dividir datos
Crear subárboles
Hoja devuelve promedio
```

10. Gradient Boosting

Complejidad

Tiempo: $O(m \cdot n \cdot d)$ Espacio: $O(m)$

Pseudocódigo (copiado estilo PDF)

```
Algoritmo GRADIENT_BOOSTING
Entrada:  $y_{\text{real}}, y_{\text{pred}}$ 
Complejidad tiempo:  $O(m \cdot n \cdot d)$ 
Complejidad espacio:  $O(m)$ 

Calcular residuos
Entrenar árbol sobre residuos
Actualizar predicción
Repetir m veces
```

Fundamentos

Fundamentos

Modelo aditivo:

$$F_{m+1}(x) = F_m(x) + \eta h_m(x)$$

Residuos:

$$r = y - F_m(x)$$

Descripción

Cada árbol corrige errores del anterior.

Pseudocódigo

```
Predicción inicial = promedio
Para m estimadores:
    calcular residuos
    entrenar árbol residuos
    actualizar predicción
```

11. XGBoost Mini

Fundamentos

Peso hoja:

$$w^* = \frac{\sum g}{\sum h + \lambda}$$

Descripción

Gradient boosting con regularización.

Pseudocódigo

```
Inicializar predicción
Para cada árbol:
    calcular residuos
    entrenar árbol regularizado
    actualizar modelo
```

12. Clustering Jerárquico

Fundamentos

Single Linkage:

$$d(C1, C2) = \min_i |x_i - x_j|$$

Descripción

Fusiona clusters hasta número objetivo.

Pseudocódigo

```
Cada punto inicia cluster
Mientras clusters > k
    hallar par más cercano
    fusionar
Asignar etiquetas
```

13. Isolation Forest

Fundamentos

Score:

$$s(x) = 2^{-E[h(x)]/c(n)}$$

Anomalías: caminos cortos.

Descripción

Detecta outliers aislandolos rápidamente.

Pseudocódigo

```
Construir muchos árboles aleatorios
Para cada punto:
    medir longitud promedio camino
    calcular score anomalía
```

14. Capa Convolutiva (CNN)

Fundamentos

Convolución:

$$Salida(i,j) = \sum_{region} region * filtro$$

Descripción

Extrae patrones (bordes, texturas).

Pseudocódigo

```
Para cada filtro:  
  recorrer imagen  
  multiplicar región por kernel  
  sumar valores  
  producir feature map
```

15. Max Pooling

Fundamentos

$$out = \max(region)$$

Descripción

Reduce dimensiones conservando rasgos importantes.

Pseudocódigo

```
Dividir en ventanas  
Tomar máximo por ventana  
Construir salida reducida
```

16. Softmax

Fundamentos

$$P(k) = \frac{e^{z_k}}{\sum e^{z_j}}$$

Descripción

Convierte salidas en probabilidades multiclase.

Pseudocódigo

```
Calcular  $z = XW + b$   
Aplicar exponenciales  
Normalizar  
retornar probabilidades
```

17. Q-Learning

Fundamentos

Ecuación Bellman:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a) - Q(s,a)]$$

Descripción

Aprende política óptima prueba-error.

Pseudocódigo

```
Inicializar Q-table  
Para cada episodio:  
  elegir acción epsilon-greedy  
  ejecutar acción  
  observar recompensa  
  actualizar Q
```

MÉTRICAS (te pueden pedir incluirlas)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precisión

$$Precision = \frac{TP}{TP+FP}$$

Recall

$$Recall = \frac{TP}{TP+FN}$$

F1

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Matriz de Confusión

		Predicho	
		Pos	Neg
Real	Pos	TP	FN
Real	Neg	FP	TN

Cómo responder si cambian el input en examen

Siempre seguir esta plantilla:

1. Definir entrada nueva.
2. Identificar si es supervisado/no supervisado.
3. Escribir fórmula principal.
4. Poner pseudocódigo.
5. Ejecutar código con nuevo input.
6. Reportar:
7. Accuracy
8. Precisión
9. Matriz de confusión
10. Interpretación
11. Conclusión

Los más probables para examen (prioridad alta)

Si solo estudias primero estos:

Supervisados:

- Regresión Logística
- KNN
- Árbol de Decisión
- SVM
- Random Forest

No supervisados:

- K-Means
- DBSCAN
- Clustering Jerárquico

Adicionales que suelen preguntar teoría:

- Gradient Boosting
- XGBoost
- Q-Learning
- CNN (Convolución + Pooling + Softmax)

Frases de conclusión (para copiar en examen)

Clasificación: "Con base en Accuracy, Precisión y matriz de confusión, el modelo muestra un desempeño adecuado para clasificar los datos, aunque podría mejorarse ajustando hiperparámetros."

Clustering: "Los resultados muestran agrupamientos consistentes según proximidad/densidad, permitiendo identificar patrones y estructura en los datos."

Anomalías: "El algoritmo permitió detectar observaciones atípicas mediante aislamiento temprano, útiles para identificar posibles anomalías."

Si después quieres también te puedo preparar los 20 ejercicios tipo examen con cambios de inputs como si fueran simulador real.